AI VIBE CODING COURSE · SAMPLE PROJECT 08

Election Results Dashboard

A small open-data project for exploring results by party and geography.

CSV → Validate → Explore → Compare

Use this worked example to practice moving from a business idea to a product requirements document, functional requirements, and technical requirements.

PROJECT SIZE
Small MVP

SUGGESTED BUILD
1-2 days

DATA TYPE
Public election results

DELIVERY
Read-only web application

1 · IDEA DESCRIPTION

Make public results easier to understand

A civic-information group wants a simple web dashboard that helps residents explore an official election-results file without using a spreadsheet. A visitor can see overall totals, filter to one district or riding, compare two areas, and read where the data came from.

Primary user

A resident, student, journalist, or community volunteer who wants a quick, neutral view of published results.

Business value

Reduce manual spreadsheet work and make a public dataset easier to navigate on desktop and mobile.

The smallest useful product

- Load one cleaned election-results dataset from a local JSON file.
- Show total votes, reporting status, turnout when available, and party vote share.
- Search or filter by geography.
- Open one geography detail page and compare two geographies.
- Display source, election date, update timestamp, and data notes.

Assumed input fields

Field	Example meaning	Required?
area_id	Stable geography ID	Yes
area_name	District or riding	Yes
party_name	Reported party label	Yes
candidate_name	Reported candidate	No
votes	Valid vote count	Yes
eligible_voters	Turnout denominator	No
reporting_status	Partial or final	Yes

Important: This is a learning scenario, not an official reporting system. Use a real government or election-authority dataset for the build, preserve its labels, and show its source and limitations.

2 · PROMPT FOR CREATING A PRD

Ask for a bounded product plan

Act as a product manager. Write a concise Product Requirements Document (PRD) for a small web application called “Election Results Dashboard.”

Business scenario:
A civic-information group wants to help residents explore one official election-results dataset without using a spreadsheet. The dashboard is informational and read-only. It is not an official results system and must not predict winners.

Users:
Residents, students, journalists, and community volunteers.

Dataset assumptions:
A cleaned CSV or JSON file contains area_id, area_name, party_name, candidate_name (optional), votes, eligible_voters (optional), reporting_status, election_name, election_date, source_url, and updated_at.

MVP capabilities:
1. Overall summary with total votes and reporting status.
2. Party totals and vote-share visualization.
3. Search/filter by geography.
4. Geography detail view.
5. Side-by-side comparison of two geographies.
6. Data source, update time, and methodology notes.
7. Responsive and keyboard-accessible interface.

Constraints:
- Small student project; target 1-2 days.
- Free-tier tools only.
- One election and one dataset.
- No accounts, payments, maps, live feeds, forecasting, winner calls, or generative summaries.
- Do not invent missing values. Show “Not provided.”

Return these sections:
Overview; problem; goals; non-goals; users; assumptions; scope; user stories; success measures; risks and mitigations; acceptance criteria; future ideas.

Write specific, testable requirements. Mark every scenario assumption clearly.

Before using: Replace the dataset assumptions with the columns in the source you selected. If the source lacks turnout data, remove turnout from the MVP rather than estimating it.

3 · PROMPT FOR FUNCTIONAL REQUIREMENTS

Turn the PRD into testable behavior

Act as a senior business analyst. Using the PRD below, write functional requirements for the “Election Results Dashboard.”

[PASTE THE APPROVED PRD HERE]

Organize requirements under:
- Data loading and validation
- Overall summary
- Party results
- Geography search and filter
- Geography detail
- Geography comparison
- Source and methodology
- Responsive design and accessibility
- Empty, loading, and error states

For each requirement, provide:
- A unique ID in the format FR-01, FR-02, and so on
- Requirement statement using “The system shall…”
- Trigger or precondition
- Expected behavior
- Acceptance criterion in Given/When/Then format
- Priority: Must, Should, or Could

Rules:
- Keep the MVP to a maximum of 12 requirements.
- Do not add login, editing, maps, live feeds, forecasting, winner calls, or AI-generated political interpretation.
- Never infer missing election values.
- Preserve official labels from the selected source.
- State how ties, zero votes, missing values, and partial reporting are displayed.
- Include keyboard operation and readable chart alternatives.
- Keep requirements implementation-neutral.

How to review the output

- ☐ Every requirement maps to one PRD goal or acceptance criterion.
- ☐ Each acceptance criterion can be demonstrated manually.
- ☐ No requirement silently expands the scope.
- ☐ Missing data is displayed honestly instead of calculated or invented.
- ☐ Chart information is also available as text or a table.

4 · PROMPT FOR TECHNICAL REQUIREMENTS

Choose the simplest viable architecture

Act as a software architect mentoring beginner developers. Using the approved PRD and functional requirements below, write technical requirements for a small “Election Results Dashboard.”

[PASTE THE APPROVED PRD HERE]
[PASTE THE APPROVED FUNCTIONAL REQUIREMENTS HERE]

Preferred solution:
- React with Vite and TypeScript
- Static hosting on a free tier
- Election data preprocessed into a local JSON file
- One GitHub repository
- No backend unless a verified requirement makes it necessary

Return:
1. Architecture overview and a simple text diagram
2. Technology choices with reasons
3. Data schema and validation rules
4. Components and routes
5. Calculation rules for totals, vote share, turnout, sorting, and ties
6. Accessibility, privacy, security, performance, and browser requirements
7. Testing strategy
8. Deployment and update process
9. Definition of done
10. Explicit non-requirements

Constraints:
- Keep the design suitable for a 1-2 day student project.
- Use free-tier or open-source tools.
- No user accounts, database, secrets, live API, analytics, or server-side processing.
- Do not invent source data or political conclusions.
- Prefer native browser features and small dependencies.
- Identify any assumption that must be confirmed against the selected dataset.

Architecture test: If the proposed solution includes authentication, a database, a queue, multiple services, or a large cloud platform, ask the model to simplify it and justify every remaining component.

The MVP answers a few clear questions

Overview

The Election Results Dashboard is a neutral, read-only web application for exploring one published election-results dataset. It converts a cleaned file into an overall view, party summary, geography detail, and two-area comparison.

Problem

Public election results are often published as large tables. People who only want to answer “How did my area vote?” or “How do these two areas compare?” must filter and calculate manually.

Goals

- Make the dataset understandable in under two minutes.
- Keep displayed numbers traceable to the source file.
- Support desktop and mobile use.
- Make core information usable without reading a chart.

Non-goals

- Official certification or live reporting.
- Winner predictions or political commentary.
- Historical trends or multiple elections.
- Accounts, saved views, maps, or data editing.

Users and needs

User	Need	Core task
Resident	Local result	Find one area
Student	Understand shares	Compare parties
Journalist	Quick comparison	Compare two areas
Volunteer	Source clarity	Verify provenance

Assumptions to confirm

The selected source has stable geography names, non-negative vote counts, a reporting-status field, and permission for public reuse. Turnout appears only if the source provides eligible-voter counts.

Scope stays narrow and measurable

User stories

- As a visitor, I want to see overall totals so I can understand the election at a glance.
- As a resident, I want to search for my geography so I can view its local result.
- As a student, I want vote shares shown with counts so I can understand proportions and scale.
- As a journalist, I want to compare two geographies so I can identify visible differences.
- As a careful reader, I want to see source and update details so I can assess the data.

MVP acceptance criteria

- ☐ The home view displays election name, date, reporting status, total votes, and party results.
- ☐ Users can find a geography by entering part of its name.
- ☐ The detail view displays all parties reported for the selected geography.
- ☐ The comparison view shows the same measures for two selected geographies.
- ☐ Counts and percentages are consistent with the loaded dataset and documented formulas.
- ☐ Missing optional fields display “Not provided”; partial results are visibly labeled.
- ☐ Source name, source link, data update time, and methodology notes are available from every view.
- ☐ All core tasks work at 360 px width and by keyboard.

Success measures for a classroom demo

TASK COMPLETION
3 core tasks
overview, search, compare

ACCURACY
10 spot checks
match source values

QUALITY
0 critical issues
keyboard and mobile

Risks and mitigations

Risk	Mitigation
Changing columns	Validate during preprocessing
Partial results mistaken as final	Persistent status label
Charts obscure small values	Show counts and data table
Political interpretation	Use neutral, sourced language

Core behavior is explicit and testable

FR-01 **Load validated results.** On application start, the system shall load the bundled dataset. *Given* a valid file, *when* loading completes, *then* the overview appears and the record count is greater than zero. **Must**

FR-02 **Reject unusable data.** The system shall show a clear error when required fields are absent or vote values are invalid. *Given* an invalid file, *when* it is loaded, *then* no calculated results appear. **Must**

FR-03 **Show the election summary.** The system shall display election name, date, status, total valid votes, and turnout only when its denominator exists. *Given* valid data, *when* the home view opens, *then* each available value is visible. **Must**

FR-04 **Summarize parties.** The system shall aggregate votes by reported party label, calculate vote share, and sort by votes descending. *Given* party rows, *when* the summary renders, *then* each party shows count and percentage. **Must**

FR-05 **Expose chart values.** Any party chart shall have an equivalent data table or textual list. *Given* a chart, *when* a keyboard or screen-reader user navigates, *then* the same values are available. **Must**

FR-06 **Search geographies.** The system shall match geography names case-insensitively as the user types. *Given* “north,” *when* matches exist, *then* only matching options are shown. **Must**

Boundary rules: Tied vote counts share the same count without an inferred ranking label. Zero is displayed as 0. An absent optional value is “Not provided.” A partial dataset is never styled as final.

Details, comparison, and resilience complete the MVP

FR-07 **Show geography detail.** The system shall show the selected area's total votes, optional turnout, status, and party results. *Given* a selected area, *when* its route opens, *then* only that area's records determine the result. **Must**

FR-08 **Compare two geographies.** The system shall allow two distinct areas and show totals and party shares side by side. *Given* two selections, *when* Compare is chosen, *then* the same metrics appear for both. **Must**

FR-09 **Explain provenance.** The system shall display the source, source link, update timestamp, field notes, and calculation formulas. *Given* any main view, *when* Source & methodology is opened, *then* those details are readable. **Must**

FR-10 **Handle empty and failed states.** The system shall show distinct loading, no-match, no-data, and load-error messages. *Given* no search match, *when* the query changes, *then* the message clears when a match exists. **Must**

FR-11 **Support keyboard and mobile use.** The system shall keep all controls labeled, focus-visible, keyboard-operable, and usable at 360 px width. *Given* keyboard-only use, *when* completing search and compare, *then* no pointer input is required. **Must**

FR-12 **Share a view.** The system should encode selected area IDs in the URL. *Given* a detail or comparison URL, *when* it opens in a new tab, *then* the same valid selection is restored. **Should**

Traceability check

PRD capability	Requirements
Overall and party views	FR-03 to FR-05
Find and inspect an area	FR-06, FR-07
Compare areas	FR-08, FR-12
Trust and usability	FR-01, FR-02, FR-09 to FR-11

A static application is enough

Architecture

Official CSV	Validation script	Static JSON	React dashboard

A local build script validates and converts the selected official source into normalized JSON. The browser downloads that JSON and performs filtering and aggregation in memory. Static hosting serves the built files; there is no server, database, user account, or stored personal data.

Technology choices

Layer	Choice	Reason
Language	TypeScript	Safer data handling
UI	React + Vite	Small, familiar toolchain
Styling	CSS	No UI dependency needed
Data	Local JSON	Fast and transparent
Tests	Vitest + Testing Library	Unit and component coverage
Hosting	Free static host	No runtime service

Normalized data shape

```
Election { name, date, status, updatedAt, sourceName, sourceUrl, areas[] }
Area { id, name, eligibleVoters?: number, reportingStatus, results[] }
Result { partyName, candidateName?: string, votes: number }
```

Validation rules

- Required strings are trimmed and non-empty; area IDs are unique.
- Votes and optional eligible-voter counts are finite integers greater than or equal to zero.
- Duplicate area-party records fail preprocessing unless the source specification requires aggregation.
- The build fails with row-level messages; it does not silently discard invalid records.

Calculations and delivery remain transparent

Routes and components

- **/** - OverviewPage, SummaryCards, PartyResults
- **/area/:id** - AreaPage, ResultTable
- **/compare?a=&b=** - ComparePage
- **/methodology** - SourcePanel, FormulaNotes
- Shared: SearchBox, StatusBanner, ErrorState

Calculation rules

- Total votes = sum of valid result votes.
- Vote share = party votes / total votes × 100.
- Turnout = total votes / eligible voters × 100, only when the denominator is positive.
- Display percentages to one decimal; retain full precision for calculations.
- Sort by votes descending, then party name ascending for equal counts.

Quality requirements

- **Accessibility:** semantic headings, labeled controls, focus styles, table alternative, contrast suitable for normal text.
- **Privacy:** no accounts, cookies, analytics, or personal-data collection.
- **Security:** escape rendered text, pin dependencies, and never load untrusted runtime files.
- **Performance:** initial compressed assets target under 1 MB; useful content target under 2 seconds on a typical broadband connection.
- **Browsers:** current major desktop and mobile browsers.

Tests

- Unit: validation, totals, shares, turnout, sorting, ties.
- Component: search, no-match, partial-status, comparison.
- Manual: keyboard journey, 360 px layout, ten source spot checks.

Six-step student build plan

1. **Select and inspect data.** Record source, license, columns, and missing values.
2. **Normalize once.** Build the validator and produce a small JSON file.
3. **Prove one slice.** Render one area's results with correct totals and shares.
4. **Add overview and search.** Reuse tested calculations.
5. **Add comparison and states.** Cover loading, missing, partial, and error cases.
6. **Test, cite, and deploy.** Run checks, spot-check values, then publish to a free static host.

Definition of done: All Must requirements pass; ten displayed values match the source; the source and formulas are visible; keyboard and 360 px checks pass; the repository contains setup, data-update, test, and deployment instructions.